

Lista 01_Larissa_Vila_Nova

Larissa Soares Vila Nova de Barros

2025-11-07

R Markdown

This is an R Markdown document. Markdown is a simple formatting syntax for authoring HTML, PDF, and MS Word documents. For more details on using R Markdown see <http://rmarkdown.rstudio.com>.

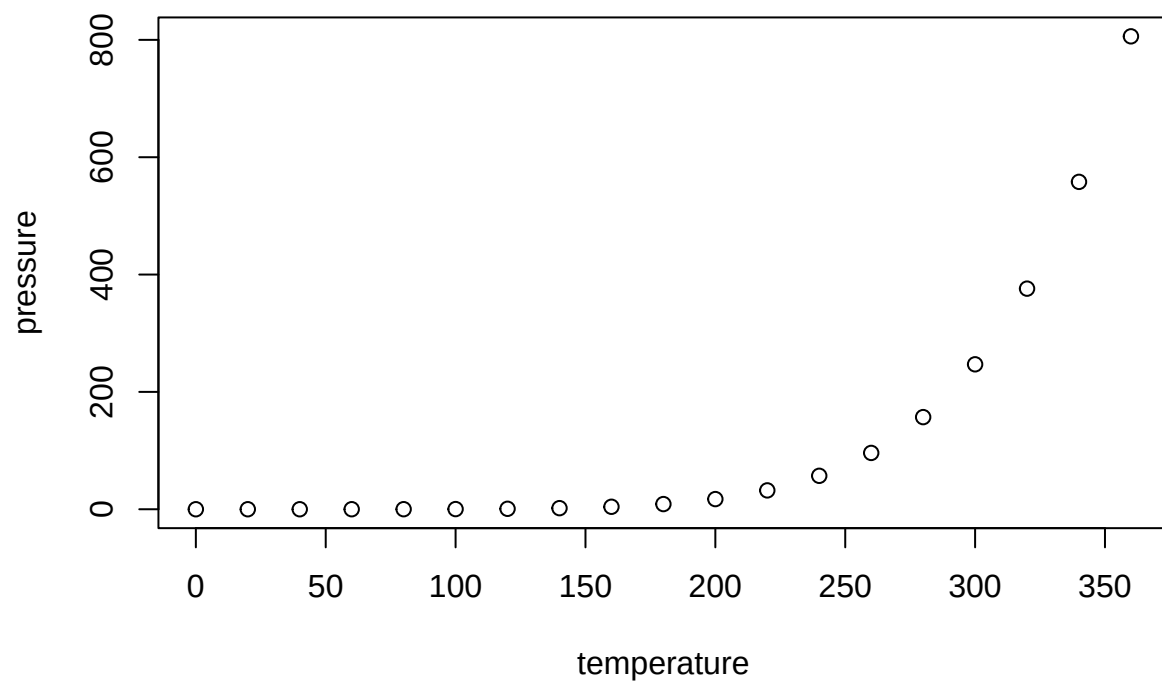
When you click the **Knit** button a document will be generated that includes both content as well as the output of any embedded R code chunks within the document. You can embed an R code chunk like this:

```
summary(cars)
```

```
##      speed      dist
##  Min.   : 4.0    Min.   :  2.00
##  1st Qu.:12.0    1st Qu.: 26.00
##  Median :15.0    Median : 36.00
##  Mean   :15.4    Mean   : 42.98
##  3rd Qu.:19.0    3rd Qu.: 56.00
##  Max.   :25.0    Max.   :120.00
```

Including Plots

You can also embed plots, for example:



Note that the `echo = FALSE` parameter was added to the code chunk to prevent printing of the R code that generated the plot.

=====

MQ101 — Métodos Quantitativos para Políticas Públicas

Lista de Exercícios #01 — TEMPLATE DE SCRIPT (R)

Preencha os campos abaixo antes de começar.

Nome: Larissa Soares Vila Nova de Barros

Matrícula/RA: _____

Turma: Métodos quantitativos 101

Data: 24/05/2025

Descrição: Respostas da Lista #01 (HOPR Partes I–II, caps. 1–8)

=====

===== ORIENTAÇÕES =====

- 1) Este script é o MODELO para a sua entrega (.R).
- 2) Execute o script de cima para baixo (Ctrl/Cmd + Shift + Enter no RStudio)

ou use o botão “Source”.
- 3) Onde estiver escrito TODO ou ENTREGA, substitua pelos seus códigos e respostas.
- 4) Use comentários explicativos: diga o que o código faz e por que isso é útil.
- 5) Use `set.seed()` quando fizer simulações para garantir reprodutibilidade.
- 6) Gere também um PDF com os resultados e interpretações (via Rmd ou outro).

7) Não utilize pacotes além de ‘ggplot2’ (opcional). Prefira base R

Se estiver no RStudio, esta chamada retorna informações do RStudio (pode falhar fora do RStudio).

```
tryCatch(RStudio.Version()$mode, error = function(e) "RStudio não detectado")
```

```
#Para instalar pacotes  
install.packages("ggplot2")
```

Reprodutibilidade global para esta lista (você pode mudar, mas mantenha constante)

```
set.seed(202501)
```

Carregamento opcional do ggplot2 (apenas se desejar usar ggplot para gráficos)

```
if (requireNamespace("ggplot2", quietly = TRUE)) { library(ggplot2) } else { message("Pacote 'ggplot2' não encontrado. Usando gráficos base R (hist).") }
```

Dica: escolha uma pasta de trabalho, se necessário (descomente e ajuste):

```
setwd("~/caminho/para/sua/pasta")
```

=====

===== **EXERCÍCIO 1 (10 pts)** =====

R como calculadora — ordem das operações (HOPR cap. 1)

Objetivo: executar expressões e entender a ordem das operações.

ENTREGA: cole os resultados no PDF e escreva, em 3–5 linhas, como os parênteses afetam o resultado.

TODO: Execute as linhas a seguir e observe os resultados.

```
10 + 2 (10 + 2) * 3 ((10 + 2) * 3 - 6) / 3
```

#Resposta #ele muda a ordem de prioridade para o calculo, alterando o resultado

=====

===== **EXERCÍCIO 2 (10 pts)**
=====

Objetos e nomeação (HOPR cap. 1)

ENTREGA: explique em 2–4 linhas a diferença entre ‘Name’ e ‘name’.

TODO: Execute e observe os resultados. R diferencia maiúsculas/minúsculas.

```
x <- 1:6 Name <- 1 name <- 0 Name + 1 name + 1
```

#Resposta: a diferença é o que ela representa. dando um objeto para cada nome, o Name representa 1 e name é 0.

=====

===== **EXERCÍCIO 3 (10 pts)**
=====

Sorteio (sample) e reprodutibilidade (HOPR cap. 1–2)

ENTREGA: descreva o papel de set.seed() em 3–5 linhas.

TODO: Compare com e sem set.seed()

```
set.seed(123) dice <- 1:6 sample(dice, size = 2, replace = TRUE)
```

#Resposta - Com o set.seed o resultado será igual com os colegas, pois ele estabelece um padrão. Se removermos o resultado fica diferente

(Teste: reexecute as linhas acima e verifique se o resultado se repete)

(Agora remova ou mude o set.seed e compare)

#Resposta: comparei deu o resultados diferentes

=====

===== EXERCÍCIO 4 (10 pts) =====

=====

Sua primeira função (HOPR cap. 1)

ENTREGA: explique o que faz cada linha da função em 4–6 linhas.

BÔNUS: implemente ‘soma3()’ (sorteia 3 números entre 1 e 6 e retorna a soma).

TODO: Defina a função e teste:

```
roll2 <- function(bones = 1:6) { # Sorteia dois valores do vetor ‘bones’ com reposição e soma. # ‘bones’  
por padrão é 1:6 (um dado comum). dice <- sample(bones, size = 2, replace = TRUE) sum(dice) }
```

#sample(...): Esta é uma função do R para amostrar aleatoriamente elementos de um vetor. #bones: A função sample vai escolher elementos do vetor que foi passado para o argumento bones. #size = 2: O sample vai sortear dois elementos. #replace = TRUE: Este é um argumento crucial. TRUE significa que a amostragem é feita com reposição. Isso permite que o mesmo número seja sorteado mais de uma vez (como em dois dados caindo no mesmo número), o que é o comportamento correto para este tipo de simulação. #dice <-: Os dois números sorteados são armazenados no objeto temporário dice.

Testes sugeridos:

```
roll2() roll2(1:20)
```

#Esta linha executa a função novamente, mas substituindo o valor padrão de bones pelo vetor 1:20. A função, então, simula o lançamento de dois dados de 20 faces, e a saída será a soma de dois números aleatórios entre 1 e 20.

TODO (BÔNUS): implementar soma3()

```
soma3 <- function() {
```

```
soma3 <- function(bones = 1:6) dice<-sample(bones, size = 3, replace = TRUE) sum(dice) #  
# }
```

=====

===== EXERCÍCIO 5 (10 pts)
=====

Ajuda e exemplos (HOPR cap. 2)

ENTREGA: resuma argumentos de `sample()` e como consultar ajuda (4–6 linhas).

TODO: Consulte a ajuda e rode exemplos

?sample example(sample)

A função `sample` gera amostras aleatórias de um vetor.

Seus argumentos principais são: `x` (o vetor de onde a amostra será retirada),

`#size` (o número de itens a serem sorteados) e `replace` (se a amostragem será com ou sem reposição, #sendo `FALSE` o padrão).

`#x` either a vector of one or more elements from which to choose, or a positive integer. See ‘Details.’ `#n` a positive number, the number of items to choose from. See ‘Details.’ `#size` a non-negative integer giving the number of items to choose. `#replace` should sampling be with replacement? `#prob` a vector of probability weights for obtaining the elements of the vector being sampled. `#useHash` logical indicating if the hash-version of the algorithm should be used. Can only be used for `replace = FALSE`, `prob = NULL`, and `size <= n/2`, and really should be used for large `n`, as `useHash=FALSE` will use memory proportional to `n`.

Dica: leia os argumentos ‘x’, ‘size’, ‘replace’, ‘prob’, etc.

=====

===== EXERCÍCIO 6 (15 pts)

=====

Simulação e histograma (HOPR cap. 1–2)

ENTREGA: histograma, média, desvio-padrão; interpretação (4–6 linhas).

Dica: você pode salvar o gráfico com `png()... dev.off()`

```
set.seed(42) somas <- replicate(10000, roll2()) length(somas) hist(somas, main = "Soma de dois dados (10.000
lançamentos)", xlab = "Soma")
mean(somas); sd(somas)
```

4. Calcular e exibir a Média e o Desvio-Padrão

```
media <- mean(somas) desvio_padrao <- sd(somas)
```

Exibe os valores arredondados para melhor visualização

```
print(paste("Média das somas:", round(media, 2))) print(paste("Desvio-padrão das somas:", round(desvio_padrao,
2)))
```

5. Criar e exibir o Histograma

```
hist(somas, main = "Soma de dois dados (10.000 lançamentos)", xlab = "Soma", ylab = "Frequência", col
= "skyblue", border = "white")
```

#Interpretação: A simulação de 10.000 lançamentos de dois dados resultou em uma distribuição aproximadamente normal e simétrica, com o formato de sino característico. O valor de soma mais frequente (moda) foi 7, o que é esperado, pois é o resultado com o maior número de combinações possíveis (6 combinações: 1+6, 2+5, 3+4, 4+3, 5+2, 6+1). A média da distribuição se situou em torno de 7.00, coincidindo com o pico central do histograma. O desvio-padrão de 2.42 indica a dispersão dos valores em torno dessa média.

(Opcional) Salvar figura:

```
png("ex6_hist_somas.png", width = 900, height = 600)
```

```
hist(somas, main = "Soma de dois dados (10.000 lançamentos)",  
xlab = "Soma")
```

```
dev.off()
```

Extensão (opcional, sem pontos extras): dado viciado favorecendo o 6

```
prob_vies <- c(rep(1/8, 5), 3/8) somas_vies <- replicate(10000, sum(sample(1:6, size = 2, replace = TRUE,  
prob = prob_vies))) # hist(somas_vies, main = "Dado viciado (6 favorecido)", xlab = "Soma")
```

=====

===== **EXERCÍCIO 7 (10 pts)**
=====

Tipos básicos (HOPR cap. 3)

ENTREGA: explique o que ‘str()’ revela sobre cada tipo e cite um uso prático.

```
dbl <- c(1.5, 2.0)  
int <- c(1L, 2L)  
chr <- c("saude", "educacao") lgl <- c(TRUE, FALSE)  
str(list(dbl = dbl, int = int, chr = chr, lgl = lgl))
```

Resposta:A função `str()` revela o tipo de dado fundamental (como `num`, `int`, `chr`, `logi`) e a dimensão (`[1:2]`) de um objeto no R. O tipo `num` (`double`) armazena valores de ponto flutuante e é ideal para cálculos estatísticos contínuos (ex: salários); o tipo `int` armazena números inteiros, útil para contagens (ex: número de filhos); o `chr` armazena texto para rótulos e nomes (ex: nomes de variáveis); e o `logi` armazena `TRUE/FALSE`, sendo essencial para filtragem e seleção de dados.

=====

===== **EXERCÍCIO 8 (10 pts)** =====

Data.frame (baralho) e mini-base municipal (HOPR cap. 3)

ENTREGA: nº de linhas/colunas e breve interpretação do `summary()`.

Baralho

```
faces <- c("ace","two","three","four","five","six","seven", "eight","nine","ten","jack","queen","king") suits
<- c("spades","hearts","diamonds","clubs") deck <- data.frame( face = rep(faces, times = 4), suit =
rep(suits, each = 13), value = rep(1:13, times = 4) ) nrow(deck); ncol(deck) head(deck, 10)
```

Mini-base municipal (dados simulados)

```
set.seed(2025) municipios <- paste0("Mun_", sprintf("%02d", 1:10)) dados_munic <- data.frame( municipio
= municipios, gasto_saude_pc = round(runif(10, 200, 1200), 2), taxa_evasao = round(runif(10, 0.00, 0.20),
3), taxa_desemprego = round(rnorm(10, 0.12, 0.03), 3) ) head(dados_munic); summary(dados_munic)
```

#O DataFrame `dados_munic` possui 10 linhas e 4 colunas. O `summary()` é essencial para a inspeção inicial: ele confirma que as taxas de #desemprego e evasão simuladas estão centradas, respectivamente, em torno de suas médias (12% para desemprego), e mostra a amplitude #de variação para o gasto em saúde per capita, sendo estas as informações descritivas básicas para iniciar qualquer análise exploratória.

=====

===== EXERCÍCIO 9 (10 pts) =====

=====

Seleção e filtros (HOPR cap. 4)

ENTREGA: descreva os retornos e quantos municípios têm `taxa_evasao > 0.10`.

```
deck[1, ] deck[c(1,3,5), c("face","suit")] deck[-(1:48), ]
subset_hearts <- deck[deck$suit == "hearts", ] nrow(subset_hearts)
evaz_alta <- dados_munic[dados_munic$taxa_evasao > 0.10, ] evaz_alta nrow(evaz_alta)
#Os municípios com retorno foram 4: o 02,03,06 e o 09, eles tiveram taxa de evasão acima de 0.10
```

=====

===== EXERCÍCIO 10 (10 pts) =====

=====

Modificando valores e NA (HOPR cap. 5)

ENTREGA: explique o efeito de `na.rm = TRUE` e quando usá-lo.

Modificando valores (ases = 14)

```
deck2 <- deck deck2$value[c(13, 26, 39, 52)] <- 14 head(deck2, 13)
```

Valores ausentes

```
vals <- c(NA, 1:5) mean(vals)
mean(vals, na.rm = TRUE)

dados_m2 <- dados_munic dados_m2$taxa_evasao[3] <- NA dados_m2$gasto_saude_pc[7] <- NA
mean(dados_m2$taxa_evasao) mean(dados_m2$taxa_evasao, na.rm=TRUE)
```

O efeito NA retorna quando há algum dado sem informação é importante usa-lo para saber se existe algum erro no R que ele não identifique.

#o definir na.rm = TRUE, você está explicitamente informando ao R para remover (ignorar) todas as observações com valores NA apenas para #aquela variável e, em seguida, calcular a estatística com os dados válidos restantes.

=====

===== EXERCÍCIO 11 (OPCIONAL, até 10 pts)
=====

Funções que "guardam estado" (HOPR cap. 6)

ENTREGA: explique o conceito e dê exemplo análogo em PP/CS.

```
setup <- function(deck_init) { DECK <- deck_init
```

```
DEAL <- function() { # Devolve a primeira carta e atualiza o baralho interno removendo-a. card <-  
DECK[1, , drop = FALSE] DECK «- DECK[-1, , drop = FALSE] return(card) }
```